

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6
«Введение в СУБД MongoDB. Установка MongoDB. Работа с БД в СУБД MongoDB»
по дисциплине «Проектирование и реализация баз данных»**

Обучающийся Ашур Амир Куссаевич

Факультет прикладной информатики

Группа K3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург

2026

Цель работы

Овладеть практическими навыками установки СУБД MongoDB, работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Оборудование

Компьютерный класс (MacBook Air, macOS).

Программное обеспечение

СУБД MongoDB 8.2, MongoDB Shell (mongosh) 2.8.3, Terminal (zsh).

ЛАБОРАТОРНАЯ РАБОТА 6.1 — Установка MongoDB и начало работы с БД

Практическое задание

- Установить MongoDB.
- Проверить работоспособность системы запуском клиента mongo.
- Выполнить методы db.help(), db.help, db.stats().
- Создать БД learn, получить список доступных БД.
- Создать коллекцию unicorns, вставив документ {name: 'Aurora', gender: 'f', weight: 450}.
- Просмотреть список коллекций, переименовать коллекцию, просмотреть статистику.
- Удалить коллекцию и БД learn.

Выполнение

Установка MongoDB

Первая попытка запуска mongosh показала, что MongoDB не установлен:

```
amish88r@Air-Amir ~ % mongosh
zsh: command not found: mongosh
```

```
amish88r — -zsh — 80x24
Last login: Thu May 21 21:17:30 on console
amish88r@Air-Amir ~ % mongosh
zsh: command not found: mongosh
amish88r@Air-Amir ~ %
```

Рисунок 1 — Попытка запуска mongosh без установки

Установка MongoDB произведена через пакетный менеджер Homebrew следующими командами:

```
brew tap mongodb/brew
brew install mongodb-community
brew install mongosh
brew services start mongodb-community
```

После запуска службы macOS вывела уведомление о добавлении фоновых объектов от MongoDB, Inc., и mongosh успешно подключился к серверу:

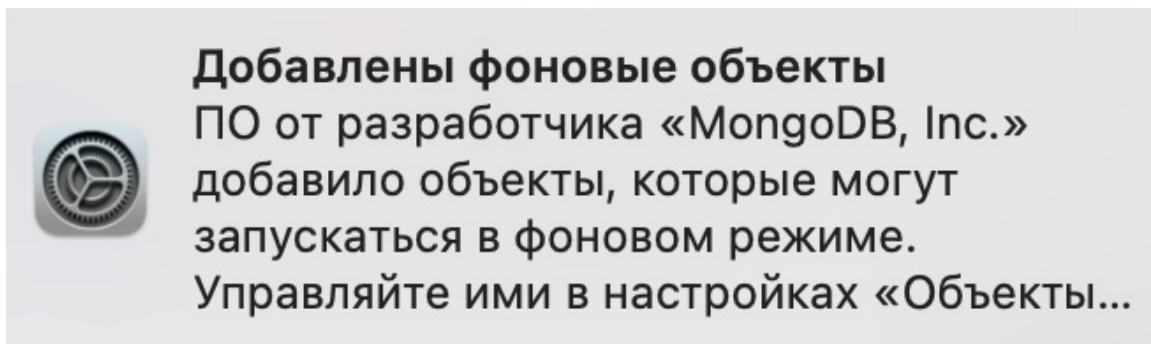


Рисунок 2 — Успешный запуск mongosh версии 2.8.3

Работа с БД — создание learn, коллекция unicorns

Выполнены команды переключения на БД learn, получения списка БД, вставки документа в коллекцию unicorns и просмотра списка коллекций:

```
use learn
show dbs
db.unicorns.insertOne({ name: 'Aurora', gender: 'f', weight: 450 })
show collections
```

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe946'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe947'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe948'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
].
```

Рисунок 3 — Создание БД learn, вставка документа, проверка коллекций

Замечание: метод insert() в MongoDB 8.x устарел и заменён на insertOne() / insertMany().

Переименование, статистика, удаление коллекции и БД

```
db.unicorns.renameCollection("unicorns_renamed")
db.unicorns_renamed.stats()
db.unicorns_renamed.drop()
db.dropDatabase()

{
  _id: ObjectId('6a0f9dac5aabc092b1bfe947'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}

]
learn> █
```

Терми

Рисунок 4 — Переименование коллекции и вывод статистики (фрагмент)

Удаление коллекции вернуло true, удаление БД вернуло { ok: 1, dropped: 'learn' }, что подтверждает успешное выполнение всех операций.

ЛАБОРАТОРНАЯ РАБОТА 6.2 — Работа с БД в СУБД MongoDB

Раздел 2.1 — Вставка документов в коллекцию

Задание 2.1.1: создать БД learn, заполнить коллекцию unicorns 11 документами; вторым способом вставить документ Dupx; проверить содержимое.

Команда вставки 11 единорогов методом insertMany:

```
use learn
db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm',
    vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f',
    vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm',
    vampires: 182},
  {name: 'Roooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires:
    99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550,
    gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f',
    vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm',
    vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm',
    vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f',
    vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm',
    vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
])
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0f9dac5aabc092b1bfe946'),
    '1': ObjectId('6a0f9dac5aabc092b1bfe947'),
    '2': ObjectId('6a0f9dac5aabc092b1bfe948'),
    '3': ObjectId('6a0f9dac5aabc092b1bfe949'),
    '4': ObjectId('6a0f9dac5aabc092b1bfe94a'),
    '5': ObjectId('6a0f9dac5aabc092b1bfe94b'),
    '6': ObjectId('6a0f9dac5aabc092b1bfe94c'),
    '7': ObjectId('6a0f9dac5aabc092b1bfe94d'),
    '8': ObjectId('6a0f9dac5aabc092b1bfe94e'),
    '9': ObjectId('6a0f9dac5aabc092b1bfe94f'),
    '10': ObjectId('6a0f9dac5aabc092b1bfe950')
  }
}
learn> █
```

Рисунок 5 — Результат вставки 11 документов (*acknowledged: true*)

Вставка двенадцатого документа через переменную (второй способ):

```
document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender:
'm', vampires: 165}
db.unicorns.insertOne(document)
db.unicorns.find()

learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe946'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe947'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe948'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
].
```

Рисунок 6 — Содержимое коллекции *unicorns* после всех вставок

Раздел 2.2 — Выборка данных

Задание 2.2.1 — самцы и самки, сортировка, ограничение

Сформировать запросы для вывода списков самцов и самок единорогов. Ограничить список самок первыми тремя особями. Отсортировать по имени. Найти всех самок, любящих carrot, ограничить findOne и limit(1).

```
db.unicorns.find({gender: 'm'}).sort({name: 1})
db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
db.unicorns.findOne({gender: 'f', loves: 'carrot'})
db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
```

```

{
  _id: ObjectId('6a0f9dac5aabc092b1bfe947'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
]
learn>

```

Терми

Рисунок 7 — Самка Aurora, найденная через findOne (любит carrot)

Задание 2.2.2 — самцы без полей loves и gender

Модифицировать запрос для вывода самцов, исключив информацию о предпочтениях и поле.

```
db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
```

```

]
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe946'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe948'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe949'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94c'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94d'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94f'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  }
]
learn>

```

Visual Studi

Рисунок 8 — Самцы без полей loves и gender

Задание 2.2.3 — обратный порядок добавления

Вывести список единорогов в обратном порядке добавления.

```
db.unicorns.find().sort({$natural: -1})
```

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe950'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94f'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94e'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94d'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94c'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94b'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94a'),
    name: 'Arya',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  }
]
```

Рисунок 9 — Единороги в обратном порядке добавления (sort по \$natural: -1)

Задание 2.1.4 — первое предпочтение, без _id

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

```
db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
```



```

]
learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] }
]
learn> █

```

Рисунок 10 — Первое предпочтение каждого единорога (оператор \$slice)

Раздел 2.3 — Логические операторы

Задание 2.3.1 — самки от 500 до 700 кг

Вывести список самок весом от полутонны до 700 кг, исключив идентификатор.

```

db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
]
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> █

```

Рисунок 11 — Самки 500–700 кг (Solnara, Leia, Nimue)

Задание 2.3.2 — самцы от 500 кг, любят grape И lemon

Вывести самцов весом от полутонны и предпочитающих grape и lemon, исключив идентификатор.

```

db.unicorns.find(
  {gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}},
  {_id: 0}
)

```

```

]
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn>

```

Рисунок 12 — Самцы от 500 кг, любящие одновременно grape и lemon (Kenny)

Задание 2.3.3 — единороги без ключа vampires

Найти всех единорогов, не имеющих ключ vampires.

```

db.unicorns.find({vampires: {$exists: false}})

learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe950'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>

```

Рисунок 13 — Единственный единорог без поля vampires (Nimue)

Задание 2.3.4 — упорядоченные самцы с первым предпочтением

Вывести упорядоченный список имён самцов с информацией об их первом предпочтении.

```

db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}})
.sort({name: 1})

learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, loves: {$slice: 1}}).sort({name: 1})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn>

```

Рисунок 14 — Самцы с первым предпочтением, отсортированные по имени

Раздел 3.1 — Запросы к вложенным объектам, курсоры

Задание 3.1.1 — коллекция towns, мэры

Создать коллекцию towns с тремя городами; вывести города с независимыми мэрами (party="I") и города с беспартийными мэрами (поле party отсутствует).

```

db.towns.insertMany([
  {name: 'Punxsutawney', population: 6200,
    last_sensus: ISODate('2008-01-31'),
    famous_for: ['phil the groundhog'],
    mayor: {name: 'Jim Wehrle'}},
  {name: 'New York', population: 22200000,
    last_sensus: ISODate('2009-07-31'),
    famous_for: ['status of liberty', 'food'],
    mayor: {name: 'Michael Bloomberg', party: 'I'}},
  {name: 'Portland', population: 528000,
    last_sensus: ISODate('2009-07-20'),
    famous_for: ['beer', 'food'],
    mayor: {name: 'Sam Adams', party: 'D'}}
])

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0f9ea55aabc092b1bfe951'),
    '1': ObjectId('6a0f9ea55aabc092b1bfe952'),
    '2': ObjectId('6a0f9ea55aabc092b1bfe953')
  }
}
learn>

```

Рисунок 15 — Создание коллекции towns (3 документа)

Города с независимыми мэрами (оператор «точка» для вложенного поля):

```

db.towns.find({'mayor.party': 'I'}, {_id: 0, name: 1, mayor: 1})
[
  {
    _id: ObjectId('6a0f9ea55aabc092b1bfe952'),
    name: 'New York',
    mayor: {
      name: 'Michael Bloomberg',
      party: 'I'
    }
  }
]
learn>

```

Рисунок 16 — Город с независимым мэром (New York, Michael Bloomberg)

Беспартийные мэры — поле party отсутствует:

```

db.towns.find({'mayor.party': {$exists: false}}, {_id: 0, name: 1, mayor: 1})
[
  {
    _id: ObjectId('6a0f9ea55aabc092b1bfe951'),
    name: 'Punxsutawney',
    mayor: {
      name: 'Jim Wehrle'
    }
  }
]
learn>

```

Рисунок 17 — Беспартийный мэр (Punxsutawney, Jim Wehrle)

Задание 3.1.2 — функция и курсор для вывода самцов

Сформировать функцию для вывода самцов; создать курсор с первыми двумя самцами с сортировкой по имени; вывести через *forEach*.

```

fn = function() { return this.gender == 'm'; }

```

```

db.unicorns.find(fn)

var cursor = db.unicorns.find({gender: 'm'}); null;
cursor.sort({name: 1}).limit(2); null;
cursor.forEach(function(obj) { print(obj.name); })

learn> db.towns.find({"mayor.party": {$exists: false}}, {_id: 0, name: 1, mayor: 1})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> // Функция для вывода самцов
| fn = function() { return this.gender == 'm'; }
| db.unicorns.find(fn)
|
| // Курсор: первые два самца, сортировка по имени, вывод через forEach
| var cursor = db.unicorns.find({gender: 'm'});null;
| cursor.sort({name: 1}).limit(2);null;
| cursor.forEach(function(obj){ print(obj.name); })
Horny
Kenny

```

Рисунок 18 — Функция + курсор + forEach: вывод «Horny, Kenny»

Раздел 3.2 — Агрегированные запросы

Задание 3.2.1 — count самок 500–600 кг

```

db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()

learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()
(node:9164) [MONGODB DRIVER] Warning: cursor.count is deprecated and will be removed in the next major version, please use `collection.estimatedDocumentCount` or `collection.countDocuments` instead
(Use `node --trace-warnings ...` to show where the warning was created)
2
learn>

```

Рисунок 19 — Количество самок весом 500–600 кг: 2

Примечание: mongosh 2.x выводит предупреждение об устаревании `cursor.count`, рекомендуя `countDocuments()`. Результат корректный.

Задание 3.2.2 — distinct предпочтений

```

db.unicorns.distinct("loves")

2
learn> db.unicorns.distinct("loves")
[
  'apple',
  'chocolate',
  'grape',
  'papaya',
  'strawberry',
  'watermelon',
  'carrot',
  'energon',
  'lemon',
  'redbull',
  'sugar'
]
learn>

```

Рисунок 20 — Уникальные предпочтения: *apple, carrot, chocolate, energon, grape, lemon, papaya, redbull, strawberry, sugar, watermelon*

Задание 3.2.3 — aggregate по полу

```
db.unicorns.aggregate([{"$group": {_id: "$gender", count: {$sum: 1}}}]])
[learn> db.unicorns.aggregate([{"$group": {_id: "$gender", count: {$sum: 1}}}]])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 6 } ]
learn> █
```

Рисунок 21 — Группировка: самок (f) — 5, самцов (m) — 6

Раздел 3.3 — Редактирование данных

Задание 3.3.1 — save Barny

Выполнить `db.unicorns.save({name: 'Barny', ...})`, проверить содержимое.

Метод `save()` в `mongosh 2.x` более не поддерживается — выдаёт ошибку `TypeError: db.unicorns.save is not a function`. Метод заменён на `insertOne()`/`replaceOne()`. Использован `insertOne`:

```
[ ] db.unicorns.find({name: 'Barny'})
TypeError: db.unicorns.save is not a function
learn> db.unicorns.save({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})
[ ] db.unicorns.find({name: 'Barny'})
TypeError: db.unicorns.save is not a function
learn> █
```

Рисунок 22 — Ошибка устаревшего метода `save`

```
db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})
db.unicorns.find({name: 'Barny'})
```

```
learn> db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})
[ ] db.unicorns.find({name: 'Barny'})
[
  {
    _id: ObjectId('6a0f9fd45aabc092b1bfe954'),
    name: 'Barny',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

Рисунок 23 — Документ *Barny* добавлен через `insertOne`

Задание 3.3.2 — обновление Айна (вес 800, vampires 51)

```
db.unicorns.updateOne({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
db.unicorns.find({name: 'Ayna'})
```

```
learn> db.unicorns.updateOne({name: 'Ayna'}, {$set: {weight: 800, vampires: 51}})
[ db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94b'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
- ]
```

Рисунок 24 — Ayna: weight: 800, vampires: 51

Задание 3.3.3 — Raleigh теперь любит redbull

```
db.unicorns.updateOne({name: 'Raleigh'}, {$push: {loves: 'redbull'}})
db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94d'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> █
```

Рисунок 25 — Raleigh: loves [apple, sugar, redbull]

Задание 3.3.4 — всем самцам +5 к vampires

```
db.unicorns.updateMany({gender: 'm'}, {$inc: {vampires: 5}})
db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, vampires: 1})
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 }
]
learn> █
```

Рисунок 26 — vampires всех самцов увеличены на 5

Задание 3.3.5 — Portland: мэр беспартийный (\$unset)

```
db.towns.updateOne({name: 'Portland'}, {$unset: {'mayor.party': 1}})
db.towns.find({name: 'Portland'})
```



```
learn> db.towns.updateOne({name: 'Portland'}, {$unset: {"mayor.party": 1}})
| db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a0f9ea55aabc092b1bfe953'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Рисунок 27 — Поле *mayor.party* удалено у *Portland*

Задание 3.3.6 — Pilot теперь любит chocolate

```
db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
db.unicorns.find({name: 'Pilot'})
```

```
learn> db.unicorns.updateOne({name: 'Pilot'}, {$push: {loves: 'chocolate'}})
| db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe94f'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> █
```

Visual S

Рисунок 28 — *Pilot: loves [apple, watermelon, chocolate]*

Задание 3.3.7 — Aurora теперь любит sugar и lemon (\$addToSet \$each)

```
db.unicorns.updateOne(
  {name: 'Aurora'},
  {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
)
db.unicorns.find({name: 'Aurora'})

learn> db.unicorns.updateOne({name: 'Aurora'}, {$addToSet: {loves: {$each: ['sugar', 'lemon']}}})
| db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe947'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> █
```

Рисунок 29 — *Aurora: loves [carrot, grape, sugar, lemon]*

Раздел 3.4 — Удаление данных

Удалить документы с беспартийными мэрами; проверить; очистить коллекцию; вывести список коллекций.

```
db.towns.deleteMany({'mayor.party': {$exists: false}})
db.towns.find()
```

```
db.towns.deleteMany({})
show collections
```

```
learn> db.unicorns.insertOne({name: 'Barny', loves: ['grape'], weight: 340, gender: 'm'})
[| db.unicorns.find({name: 'Barny'})
[
  {
    _id: ObjectId('6a0f9fd45aabc092b1bfe954'),
    name: 'Barny',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 2 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a0f9ea55aabc092b1bfe952'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 1 }
learn> show collections
towns
unicorns
learn> █
```

Рисунок 30 — Удаление беспартийных (*deletedCount: 2* — *Punxsutawney* и *Portland*, у которого *party* удалили в 3.3.5), очистка коллекции, список оставшихся коллекций

Раздел 4.1 — Ссылки между коллекциями (DBRef)

Создать коллекцию зон обитания единорогов; включить ссылки на зону для нескольких единорогов вторым способом (DBRef).

```
db.zones.insertMany([
  { _id: 'forest', full_name: 'Лесная зона', description: 'Густые леса' },
  { _id: 'meadow', full_name: 'Луговая зона', description: 'Открытые
поляны' },
  { _id: 'mountain', full_name: 'Горная зона', description: 'Высокогорные
районы' }
])

db.unicorns.updateOne({name: 'Horny'},
{$set: {zone: {$ref: 'zones', $id: 'forest'}}})
db.unicorns.updateOne({name: 'Aurora'},
{$set: {zone: {$ref: 'zones', $id: 'meadow'}}})
db.unicorns.updateOne({name: 'Unicrom'}, {$set: {zone: {$ref: 'zones', $id: 'mountai
n'}}})

db.unicorns.find({name: 'Horny'})
```



```

learn> db.zones.insertMany([
|   { _id: "forest", full_name: "Лесная зона", description: "Густые леса"},
|   { _id: "meadow", full_name: "Луговая зона", description: "Открытые поляны"},
|   { _id: "mountain", full_name: "Горная зона", description: "Высокогорные районы"}
| ])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'meadow', '2': 'mountain' }
}
learn> db.unicorns.updateOne({name: 'Horny'}, {$set: {zone: {$ref: "zones", $id: "forest"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne({name: 'Aurora'}, {$set: {zone: {$ref: "zones", $id: "meadow"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne({name: 'Unicrom'}, {$set: {zone: {$ref: "zones", $id: "mountain"}}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Horny'})
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe946'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68,
    zone: DBRef('zones', 'forest')
  }
]
learn> var u = db.unicorns.findOne({name: 'Horny'})
[] db.zones.findOne({_id: u.zone.$id})
null
learn>

```

Рисунок 31 — Создание коллекции *zones* и установка *DBRef*-ссылок; в документе *Horny* видно поле *zone: DBRef('zones', 'forest')*

Разделы 4.2–4.3 — Настройка и управление индексами

Задание 4.2.1 — уникальный индекс по *name*

```

db.unicorns.createIndex({name: 1}, {unique: true})
db.unicorns.getIndexes()

```

```

}
learn> db.unicorns.find({name: 'Horny'})
[
  {
    _id: ObjectId('6a0f9dac5aabc092b1bfe946'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68,
    zone: DBRef('zones', 'forest')
  }
]
learn> var u = db.unicorns.findOne({name: 'Horny'})
| db.zones.findOne({_id: u.zone.$id})
null
learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> █

```

Рисунок 32 — Создан индекс name_1 с unique: true; getIndexes() возвращает _id_ и name_1

Уникальный индекс по полю name создан успешно (так как все имена единорогов уникальны).

Задание 4.3.1 — удаление индексов

```

db.unicorns.dropIndex("name_1")
db.unicorns.dropIndex("_id_")

}
]
learn> var u = db.unicorns.findOne({name: 'Horny'})
| db.zones.findOne({_id: u.zone.$id})
null
[learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
[learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
[learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
[learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> █

```

Рисунок 33 — `dropIndex("name_1") → { nIndexesWas: 2, ok: 1 }`; `dropIndex("_id_") → ошибка cannot drop _id index`

Индекс `_id_` удалить невозможно — это системный индекс, создаваемый автоматически и обязательный для работы коллекции.

Раздел 4.4 — План запроса (explain)

Создать объёмную коллекцию `numbers` (100000 документов), выбрать последние 4, проанализировать план запроса до и после создания индекса по `value`.

```
for(i = 0; i < 100000; i++){ db.numbers.insertOne({value: i}) }
db.numbers.find().sort({$natural: -1}).limit(4)
db.numbers.explain('executionStats').find().sort({$natural: -1}).limit(4)
```

```
learn> db.numbers.find().sort({$natural: -1}).limit(4)
[
  { _id: ObjectId('6a0fa0e75aabc092b1c28a88'), value: 75432 },
  { _id: ObjectId('6a0fa0e75aabc092b1c28a87'), value: 75431 },
  { _id: ObjectId('6a0fa0e75aabc092b1c28a86'), value: 75430 },
  { _id: ObjectId('6a0fa0e75aabc092b1c28a85'), value: 75429 }
]
learn> db.numbers.explain("executionStats").find().sort({$natural: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: '4E45B701',
    planCacheShapeHash: '4E45B701',
    planCacheKey: '36FF8BFB',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: { stage: 'COLLSCAN', direction: 'backward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      limitAmount: 4,
      inputStage: {
        stage: 'COLLSCAN',
        nReturned: 4,
        executionTimeMillisEstimate: 0,
        works: 4,

```

Рисунок 34 — План запроса БЕЗ индекса: `stage COLLSCAN backward`, `executionTimeMillis: 0`, `totalDocsExamined: 4`

Далее создан индекс по полю `value` и запрос выполнен повторно:

```
db.numbers.createIndex({value: 1})
db.numbers.getIndexes()
db.numbers.explain('executionStats').find().sort({$natural: -1}).limit(4)
```

```

    }
  }
}
[learn> db.numbers.createIndex({value: 1})
value_1
[learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_',
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
[learn> db.numbers.explain("executionStats").find().sort({$natural: -1}).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: '4E45B701',
    planCacheShapeHash: '4E45B701',
    planCacheKey: '36FF8BFB',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: { stage: 'COLLSCAN', direction: 'backward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 4,
    executionStages: {
      isCached: false,
      stage: 'LIMIT'
    }
  }
}

```

Рисунок 35 — План запроса *C* индексом *value_1*: stage по-прежнему COLLSCAN, *executionTimeMillis*: 0

Анализ результатов

Время выполнения в обоих случаях: 0 мс; количество просмотренных документов: 4 (*totalDocsExamined*: 4).

Индекс по полю *value* не повлиял на план запроса, поскольку запрос использует сортировку по физическому порядку (*\$natural*: -1) и *limit*(4). MongoDB читает последние 4 документа коллекции в обратном порядке прямой проход (COLLSCAN backward) без полного сканирования — это и так оптимально для данного запроса.

Индекс по *value* был бы эффективен для запросов с фильтром или сортировкой по самому полю *value*, например: *db.numbers.find().sort({value: -1}).limit(4)* — в этом случае использовался бы IXSCAN.

Ответы на контрольные вопросы

1. Как используется оператор «точка»?

Оператор «точка» используется для обращения к полям вложенных документов. Например, `db.towns.find( {"mayor.party": "I" } )` ищет документы, у которых внутри вложенного объекта `mayor` поле `party` равно "I".

2. Как можно использовать курсор?

Курсор — это результат выполнения `find()`, инкапсулирующий набор документов. Курсор позволяет применять цепочки методов `sort()`, `limit()`, `skip()`, а также перебирать документы через `hasNext()/next()` или `forEach()`. Это даёт возможность обрабатывать результаты выборки программно и не загружать все документы сразу.

3. Какие возможности агрегирования данных существуют в MongoDB?

MongoDB предоставляет три механизма агрегации: метод `count()` (теперь `countDocuments()` / `estimatedDocumentCount()`) — подсчёт документов; `distinct()` — получение уникальных значений поля; `aggregate()` — построение конвейера (pipeline) с операторами `$group`, `$match`, `$sort`, `$project`, `$sum`, `$avg` и др. — это самый мощный механизм, аналог GROUP BY в SQL.

4. Какая из функций `save` или `update` более детально позволит настроить редактирование документов коллекции?

Более детальную настройку даёт `update` (`updateOne` / `updateMany`), которая принимает условие выборки, объект изменений с операторами (`$set`, `$inc`, `$push`, `$unset`, `$addToSet` и др.) и опции (`upsert`, `multi`). `save` был упрощённой обёрткой и в современных версиях MongoDB удалён.

5. Как происходит удаление документов из коллекции по умолчанию?

По умолчанию метод `remove` (а в современных версиях `deleteMany`) удаляет все документы, соответствующие условию выборки. Чтобы удалить только один документ, используется `deleteOne()`. Пустой фильтр `{}` удаляет все документы коллекции.

6. Назовите способы связывания коллекций в MongoDB.

Ручная установка ссылок (запись `_id` одного документа в поле другого) и автоматическое связывание через `DBRef` (объект с полями `$ref` — имя коллекции, `$id` — идентификатор, опционально `$db` — имя БД).

7. Сколько индексов можно установить на одну коллекцию в БД MongoDB?

Максимум 64 индекса на одну коллекцию.

8. Как получить информацию обо всех индексах базы данных MongoDB?

Через метод `db.имя_коллекции.getIndexes()`, который возвращает массив с описанием всех индексов коллекции (имя, ключи, опции).

Выводы

В ходе выполнения лабораторных работ 6.1 и 6.2 получены практические навыки установки и настройки СУБД MongoDB 8.2 на macOS через пакетный менеджер Homebrew. Изучены основные команды mongosh, отработаны все CRUD-операции (вставка, выборка, обновление, удаление документов), а также работа с вложенными объектами, курсорами, агрегацией и редактированием данных операторами \$set, \$inc, \$push, \$addToSet, \$unset.

Освоены механизмы связывания коллекций через DBRef, настройка обычных и уникальных индексов, а также анализ плана выполнения запроса методом explain("executionStats"). Установлено, что эффективность индекса зависит от типа запроса: индекс ускоряет поиск и сортировку по индексированному полю, но не влияет на запросы, использующие сортировку по физическому порядку \$natural.

Замечено, что ряд методов из методички устарел в MongoDB 8.x: insert() заменён на insertOne()/insertMany(), save() удалён, cursor.count() помечен как deprecated в пользу countDocuments(). В работе использованы актуальные аналоги.